

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

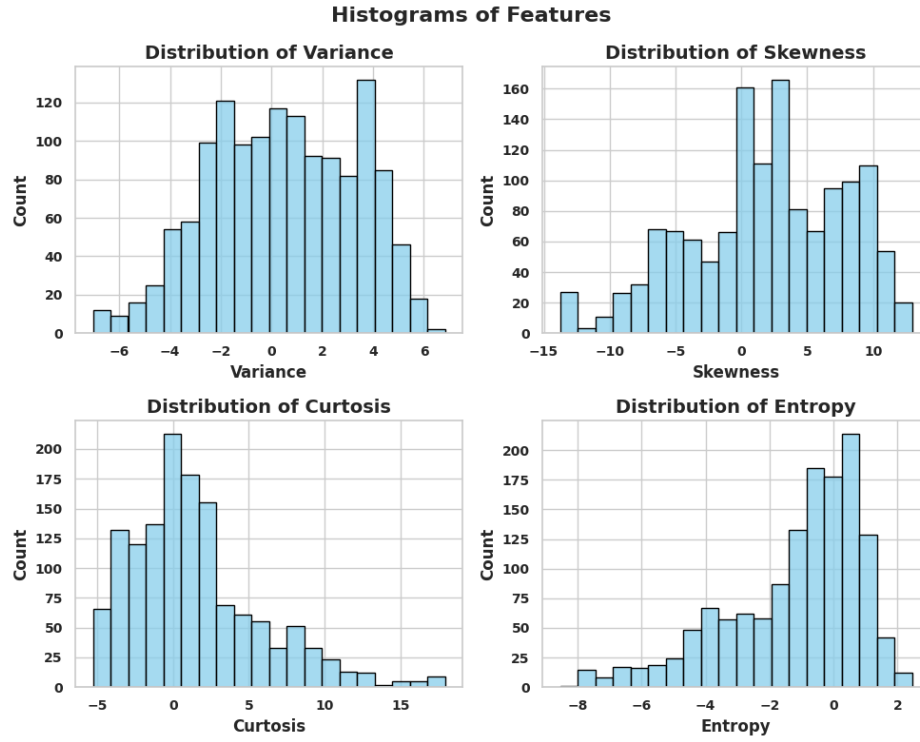
Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

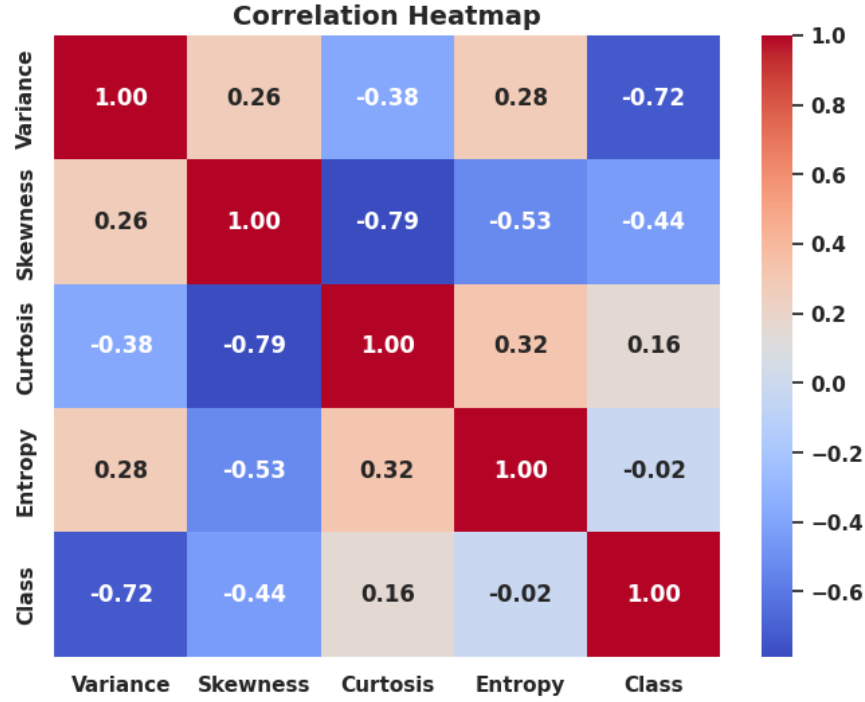
5. Source Code

The complete source code for this experiment is available at the following GitHub repository:
<https://github.com/Jai-Abhishek/DL-Lab>

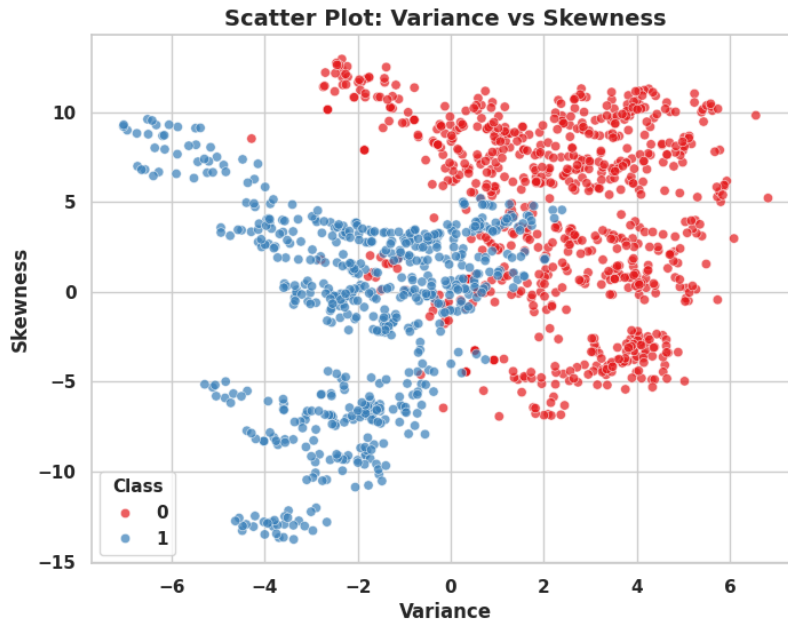
6. Generated Plots with Interpretation



The histograms suggest that while Variance and Entropy are more or less similar, with variability in modality, the histograms for Skewness and Kurtosis are significantly skewed rightwards and contain high values. The differences in shape and distribution among variables indicate that data should be normalized before training with the perceptron.

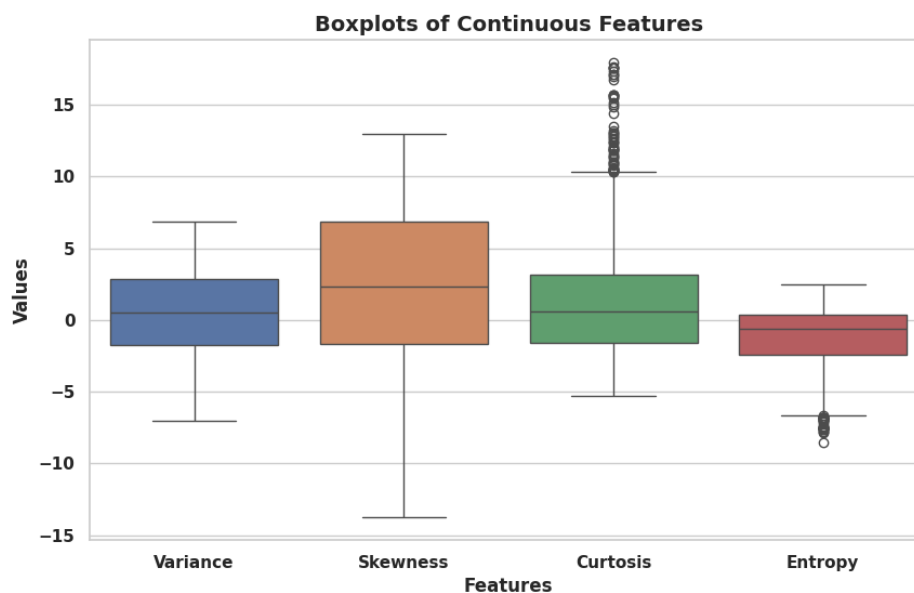


When checking correlation of the variables with the target, we see that Variance has the highest correlation (-0.72), Skewness comes next (-0.44), which indicates that among the given features, these are the most powerful for the analysis of authenticity of the documents. Skewness and Kurtosis are strongly negatively correlated too (-0.79), which makes us suspect of their redundancy.



The scatter plot of Variance and Skewness makes it clear that the classes are mostly placed in the separated areas of the feature space, with the cluster of genuine notes (Class 0) being at the higher values of both Variance and Skewness, while the cluster of counterfeit notes (Class 1) is

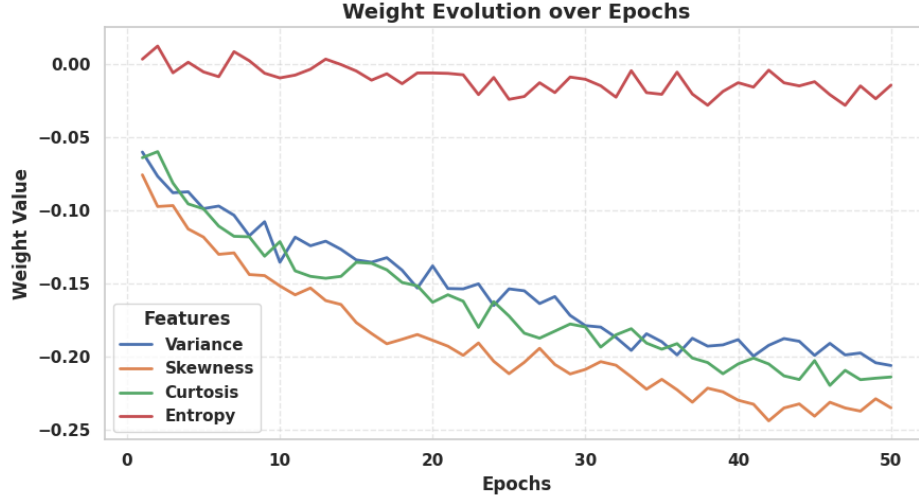
located at the lower (including negative) ends of the scale. It should be noted that there is also an area of overlap, which makes a small number of misclassifications possible.



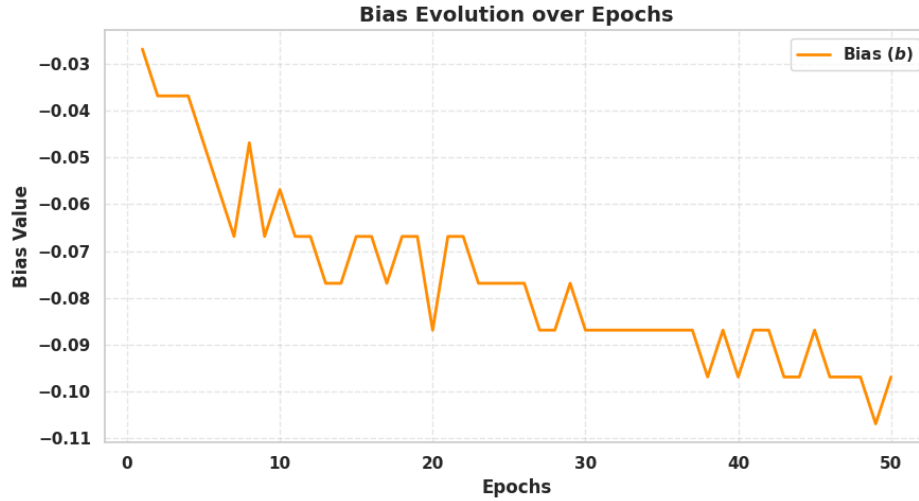
The boxplots depict that Kurtosis has a good number of outliers with high-value extremes while the Entropy has a small group of outliers with lower values than that of Kurtosis; however, variance and Skewness show no extreme outliers. Skewness has the largest interquartile range amongst the four features which is coincident with the wide and positively skewed histogram of the Skewness feature.



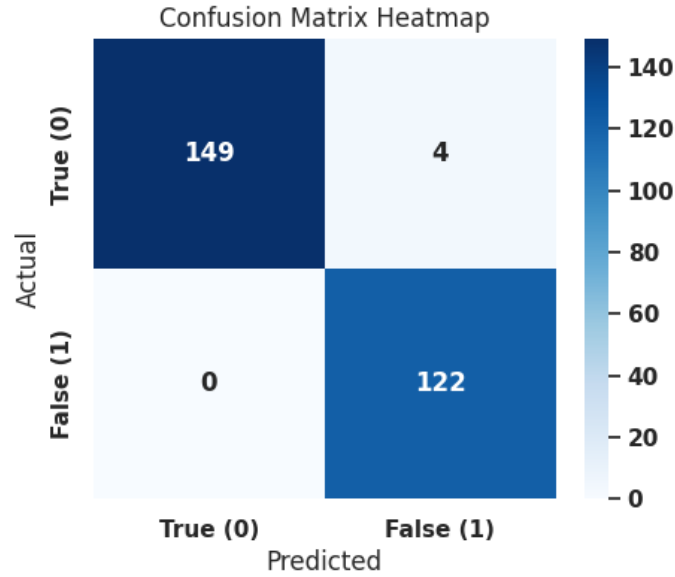
The errors in training reduce sharply from 52 to nearly 20 misclassifications in the first two epochs of training and afterward fluctuate within the small range of approximately 12 and 25 during the remaining epochs. This proves that the dataset is not perfectly linearly separable thus the perceptron cannot achieve zero error convergence, but rather ends up with a very low near-constant error rate.



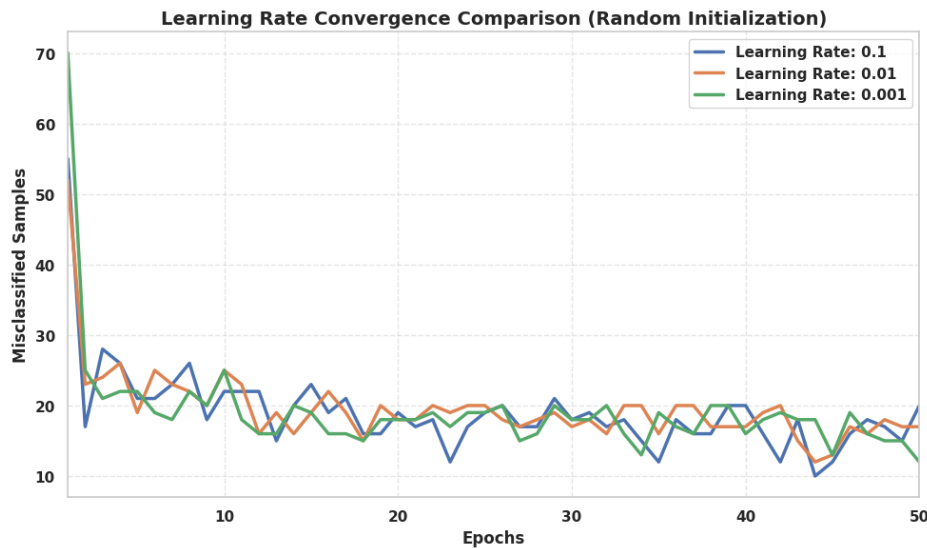
The weights of Variance, Skewness, and Kurtosis keep decreasing and turning more negative in the 50 epochs, where the size of Skewness weight reaches the largest size (approximately -0.23), indicating its important role in the decision boundary. The Entropy weight remains near to zero throughout the training process, which is also confirmed with almost zero value of its correlation (-0.02) with the target class obtained from the correlation heat map observation.



The bias is declining gradually from the value of about -0.03 to that of approximately -0.10 across the training period, following the same pattern as described for the weights. Such bias transformation goes together with the weights acquired, in order for the decision boundary position to be adjusted based on the separation of two classes.



The confusion matrix shows 149 true negatives and 122 true positives for the test data, while false positives amount just to 4, while false negatives are absent. The missing false negatives indicates that the perceptron did not fail to detect the forged banknote; on the contrary, it sometimes mistakenly classified a real banknote as forged.



All three learning rates (0.1, 0.01, and 0.001) reach a converged error range of approximately 15 to 20 misclassifications after about 15 to 20 epochs, providing evidence that the final performance of the perceptron is not very sensitive to the learning rate. The highest learning rate used (0.1) produces a bit more fluctuation in the first few epochs because of larger updates while the lowest rate (0.001) gives a smooth and slightly slower start.

7. Performance Tables

Training Summary

Dataset Size	1372 samples (5 columns)
Train/Test Split	Training: 1097 samples (80%) Testing: 275 samples (20%)
Learning Rate	0.01
Epochs	50
Final Weights	Variance: -0.2058 , Skewness: -0.2347 , Curtosis: -0.2137 , Entropy: -0.0144
Final Bias	-0.0969
Accuracy	98.5455%
Precision	96.8254%
Recall	100.0000%
F1-score	98.3871%

Epoch-wise Learning

Weight 1 is for variance, and Weight 2 is for skewness with the complete four-feature weight vector for the last epoch shown in the above training summary.

Epoch	Errors	Weight 1	Weight 2	Bias
1	52	-0.0601	-0.0756	-0.0269
10	25	-0.1353	-0.1515	-0.0569
20	18	-0.1378	-0.1886	-0.0869
30	17	-0.1786	-0.2085	-0.0869
40	17	-0.1881	-0.2295	-0.0969
50	17	-0.2058	-0.2347	-0.0969

8. Discussion and Conclusion

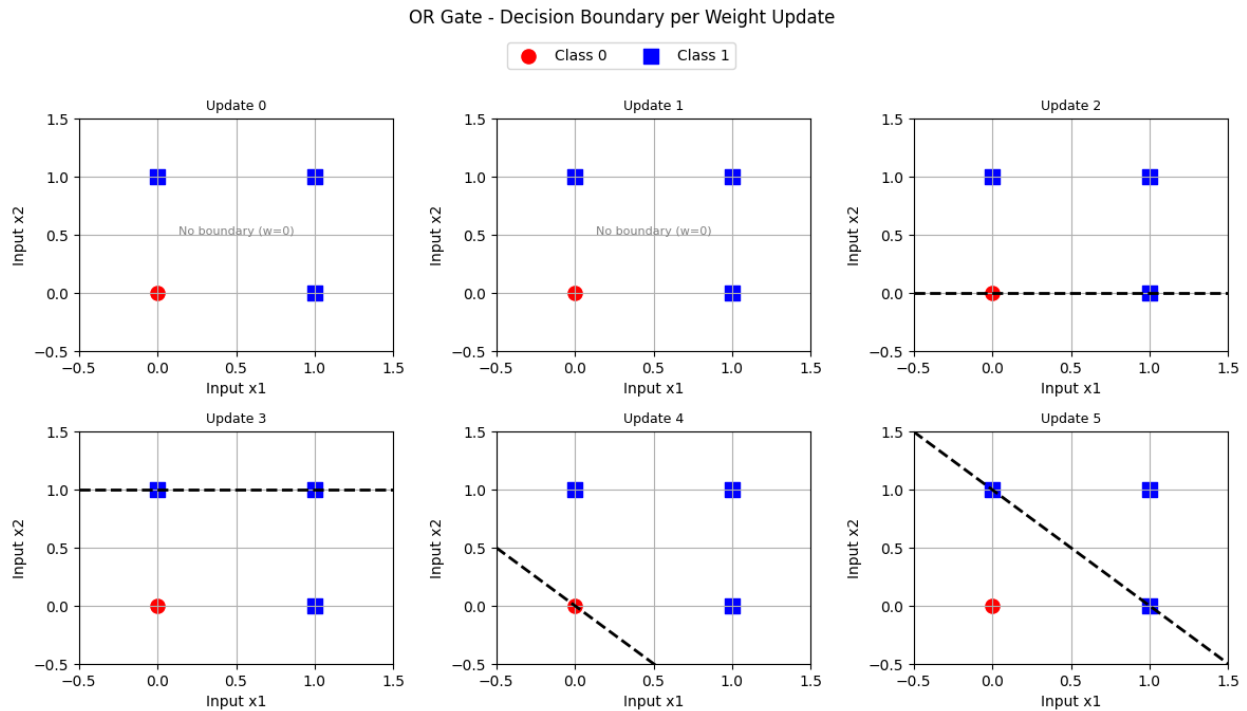
The Scratch-built Single Layer Perceptron has effectively resulted in the identification of a relatively linear decision boundary that separates real and fake banknotes with a considerable level of accuracy. Correlation analysis and scatterplots make it clear that the variance and skewness as the essential distinguishing features have been duly incorporated into the trained model since these features are reflected in the largest weights, while the entropy has appeared to remain almost the same. Moreover, the training error curve depicts how fast improvement occurred at first, yet the convergence failed to occur. This corresponds well to the overlap observed in the scatterplot.

The results obtained by this model in the unused test data indicated an accuracy of 98.55%, precision of 96.83%, recall of 100%, and F1 score of 98.39%. It is evident from the outcomes of the study that there are no false negatives for this model and only four false positives were present. It is possible to conclude that the model is able to identify counterfeit banknotes, however, from time to time it defines a banknote as counterfeit while in fact it is a real one. The overall results show that this model can be applied for tasks connected with linearly separable datasets successfully.

9. Extended Experiment: Logic Gates and the Limitation of a Single Perceptron

In an effort to show how a perceptron adheres to the method of linear separability, the same weight update was applied to four main logic gate functions (OR, AND, NOT, and XOR). For every gate, a perceptron was trained with a learning rate of $\eta = 0.1$, and the decision boundary was plotted right after every weight updating step.

9.1 OR Gate

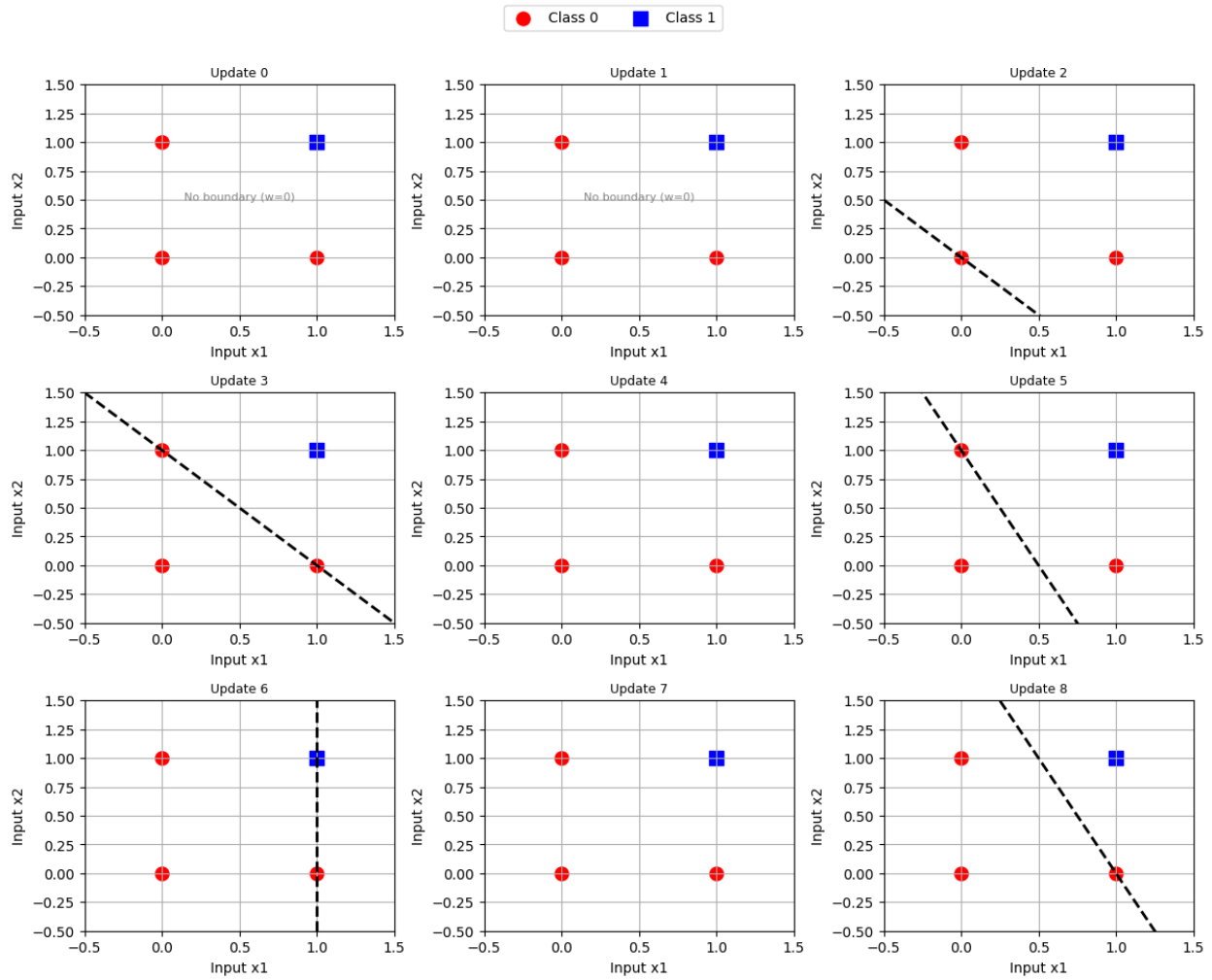


The OR gate is declared to possess linear separability, and the training was completed after 6 weight updates (2 full passes through the training set of 4 samples).

Update	Bias	Weight 1	Weight 2
0	0.0000	0.0000	0.0000
1	-0.1000	0.0000	0.0000
2	0.0000	0.0000	0.1000
3	-0.1000	0.0000	0.1000
4	0.0000	0.1000	0.1000
5 (final, converged)	-0.1000	0.1000	0.1000

9.2 AND Gate

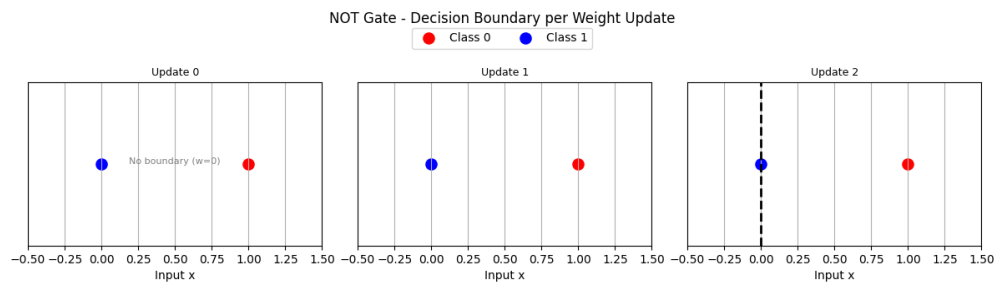
AND Gate - Decision Boundary per Weight Update



AND is another linearly separable gate but it has more weight updates compared to OR because the positive sample point (1,1) is surrounded by three negative training points which makes alterations of the separation boundary necessary until it meets the requirement. 8 weight updates are required for convergence of the perceptron for this gate.

Update	Bias	Weight 1	Weight 2
0	0.0000	0.0000	0.0000
1	-0.1000	0.0000	0.0000
2	0.0000	0.1000	0.1000
3	-0.1000	0.1000	0.1000
4	-0.2000	0.1000	0.0000
5	-0.1000	0.2000	0.1000
6	-0.2000	0.2000	0.0000
7	-0.3000	0.1000	0.0000
8 (final, converged)	-0.2000	0.2000	0.1000

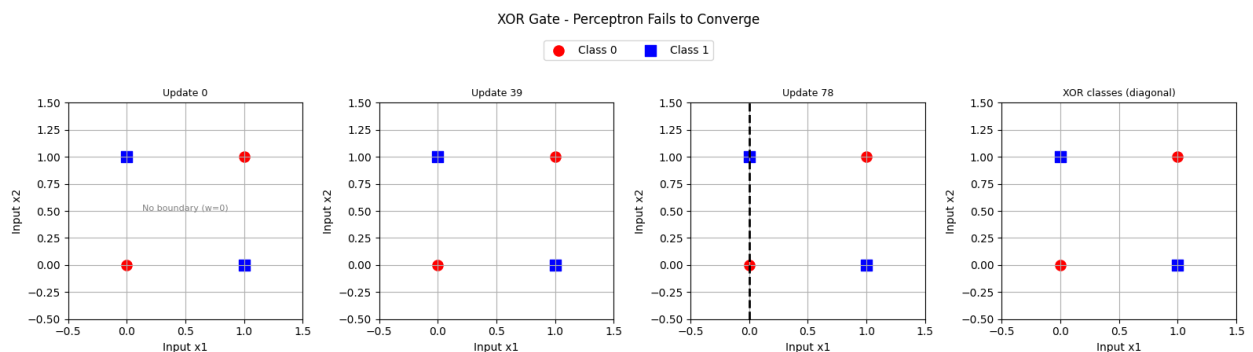
9.3 NOT Gate



The single-input NOT gate can be considered trivially linearly separable. The two inputs, $x = 0$ and $x = 1$ can be broken down into multiple states using a single threshold line. Therefore, only 2 weight updates were used for the convergence of a perceptron.

Update	Bias	Weight 1
0	0.0000	0.0000
1	-0.1000	-0.1000
2 (final, converged)	0.0000	-0.1000

9.4 XOR Gate



In contrast to the above gates, in case of their implementation using the perceptron on the input XOR gate, perceptron would never converge and would thus go through an infinite series of updates which would yield multiple weight vectors on each epoch. In our case, the perceptron was run for 20 epochs or 79 weight updates, however instead of reaching a suitable weight vector, the perceptron got stuck in the same four stages shown below from Update 3 and thus would go through an infinite series of repeating updates.

Update	Bias	Weight 1	Weight 2
0	0.0000	0.0000	0.0000
1	−0.1000	0.0000	0.0000
2	0.0000	0.0000	0.1000
3	−0.1000	−0.1000	0.0000
4	0.0000	−0.1000	0.1000
5	0.1000	0.0000	0.1000
6	0.0000	−0.1000	0.0000
7 (= Update 3)	−0.1000	−0.1000	0.0000
8 (= Update 4)	0.0000	−0.1000	0.1000
9 (= Update 5)	0.1000	0.0000	0.1000
⋮	cycle of length 4 (Updates 3–6) repeats forever		
78 (after 20 epochs, not converged)	0.0000	−0.1000	0.0000

Reason XOR Does Not Converge

The XOR truth table is $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$. The two Class-1 points $\{(0,1), (1,0)\}$ and the two Class-0 points $\{(0,0), (1,1)\}$ lie on the two opposite diagonals of the unit square, and both diagonals share the same midpoint $(0.5, 0.5)$. A perceptron can only represent a single straight line $w_1x_1 + w_2x_2 + b = 0$ as its decision boundary. Because both diagonals cross at the same centre point, any straight line that separates one pair of diagonal points must place the other pair on the same side as each other rather than splitting them correctly – so no straight line can classify all four XOR points correctly. XOR is therefore **not linearly separable**. Since the perceptron weight-update rule only terminates when it finds a weight vector that misclassifies zero training samples, and no such vector exists for XOR, the algorithm keeps making corrective updates forever, causing the weights to cycle through the same few values (as seen in the table above) instead of converging. This is the classical example (Minsky and Papert, 1969) used to demonstrate the fundamental representational limitation of a single-layer perceptron, and it motivates the use of multilayer perceptrons with a hidden layer, which can combine several linear boundaries to form the non-linear decision region XOR requires.

10. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.